

NLP Course



Search documentation

Ctrl+K

EN

2,358

0. SETUP

1. TRANSFORMER MODELS

2. USING TRANSFORMERS

Introduction

Behind the pipeline

Models

Tokenizers

Handling multiple sequences

Putting it all together

Basic usage completed!

End-of-chapter quiz

3. FINE-TUNING A PRETRAINED MODEL

4. SHARING MODELS AND TOKENIZERS

5. THE DATASETS LIBRARY

6. THE TOKENIZERS LIBRARY

7. MAIN NLP TASKS

8. HOW TO ASK FOR HELP

9. BUILDING AND SHARING DEMOS

COURSE EVENTS

Join the Hugging Face community

and get access to the augmented documentation experience

 Collaborate on models, datasets and Spaces

 Faster examples with accelerated inference

 Switch between documentation themes

Sign Up

to get started

 PyTorch

 TensorFlow

 Ask a question

 Open in Colab

 Open In Studio Lab

Putting it all together

In the last few sections, we've been trying our best to do most of the work by hand. We've explored how tokenizers work and looked at tokenization, conversion to input IDs, padding, truncation, and attention masks.

However, as we saw in section 2, the `transformers` API can handle all of this for us with a high-level function that we'll dive into here. When you call your `tokenizer` directly on the sentence, you get back inputs that are ready to pass through your model:

```
from transformers import AutoTokenizer

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

sequence = "I've been waiting for a HuggingFace course my whole life."

model_inputs = tokenizer(sequence)
```

Here, the `model_inputs` variable contains everything that's necessary for a model to operate well. For DistilBERT, that includes the input IDs as well as the attention mask. Other models that accept additional inputs will also have those output by the `tokenizer` object.

As we'll see in some examples below, this method is very powerful. First, it can tokenize a single sequence:

```
sequence = "I've been waiting for a HuggingFace course my whole life."

model_inputs = tokenizer(sequence)
```

It also handles multiple sequences at a time, with no change in the API:

```
sequences = ["I've been waiting for a HuggingFace course my whole life.", "So have I!"]

model_inputs = tokenizer(sequences)
```

It can pad according to several objectives:

```
# Will pad the sequences up to the maximum sequence length
model_inputs = tokenizer(sequences, padding="longest")

# Will pad the sequences up to the model max length
# (512 for BERT or DistilBERT)
model_inputs = tokenizer(sequences, padding="max_length")

# Will pad the sequences up to the specified max length
model_inputs = tokenizer(sequences, padding="max_length", max_length=8)
```

It can also truncate sequences:

```
sequences = ["I've been waiting for a HuggingFace course my whole life.", "So have I!"]

# Will truncate the sequences that are longer than the model max length
# (512 for BERT or DistilBERT)
model_inputs = tokenizer(sequences, truncation=True)

# Will truncate the sequences that are longer than the specified max length
model_inputs = tokenizer(sequences, max_length=8, truncation=True)
```

The `tokenizer` object can handle the conversion to specific framework tensors, which can then be directly sent to the model. For example, in the following code sample we are prompting the tokenizer to return tensors from the different frameworks — `"pt"` returns PyTorch tensors, `"tf"` returns TensorFlow tensors, and `"np"` returns NumPy arrays:

```
sequences = ["I've been waiting for a HuggingFace course my whole life.", "So have I!"]

# Returns PyTorch tensors
model_inputs = tokenizer(sequences, padding=True, return_tensors="pt")

# Returns TensorFlow tensors
model_inputs = tokenizer(sequences, padding=True, return_tensors="tf")

# Returns NumPy arrays
model_inputs = tokenizer(sequences, padding=True, return_tensors="np")
```

Special tokens

If we take a look at the input IDs returned by the tokenizer, we will see they are a tiny bit different from what we had earlier:

```
sequence = "I've been waiting for a HuggingFace course my whole life."

model_inputs = tokenizer(sequence)
print(model_inputs["input_ids"])

tokens = tokenizer.tokenize(sequence)
ids = tokenizer.convert_tokens_to_ids(tokens)
print(ids)
```

```
[101, 1045, 1005, 2310, 2042, 3403, 2005, 1037, 17662, 12172, 2607, 2026, 2878, 2166, 1012, 102]
[1045, 1005, 2310, 2042, 3403, 2005, 1037, 17662, 12172, 2607, 2026, 2878, 2166, 1012]
```

One token ID was added at the beginning, and one at the end. Let's decode the two sequences of IDs above to see what this is about:

```
print(tokenizer.decode(model_inputs["input_ids"]))
print(tokenizer.decode(ids))
```

```
"[CLS] i've been waiting for a huggingface course my whole life. [SEP]"
"i've been waiting for a huggingface course my whole life."
```

The tokenizer added the special word `[CLS]` at the beginning and the special word `[SEP]` at the end. This is because the model was pretrained with those, so to get the same results for inference we need to add them as well. Note that some models don't add special words, or add different ones; models may also add these special words only at the beginning, or only at the end. In any case, the tokenizer knows which ones are expected and will deal with this for you.

Wrapping up: From tokenizer to model

Now that we've seen all the individual steps the `tokenizer` object uses when applied on texts, let's see one final time how it can handle multiple sequences (padding!), very long sequences (truncation!), and multiple types of tensors with its main API:

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
sequences = ["I've been waiting for a HuggingFace course my whole life.", "So have I!"]

tokens = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")
output = model(**tokens)
```

Putting it all together

Special tokens

Wrapping up: From tokenizer to model

